

课程笔记

最优传输的理论与计算系列讲座之九：几何变分算法

顾险峰讲授 / 课程笔记整理

2026 年 6 月 15 日



视频作者/频道: Upupupxiaohua (Bilibili)

发布日期: 2023 年 11 月 21 日

视频时长: 1 小时 24 分钟

视频链接: <https://www.bilibili.com/video/BV1Da4y1U78N?p=9>

目录

1 引言：从理论到计算的过渡	3
1.1 课程背景与目标	3
1.2 几何编程的核心挑战：数据结构	3
1.3 工业软件与几何处理	3
1.4 本章小结	4
2 三角网格与最优传输可视化	4
2.1 三维模型展示	4
2.2 高维网格的本质困难	5
2.3 切割平面与体网格	5
2.4 最优传输的可视化	6
2.5 三角剖分的自适应性	6
2.6 本章小结	7
3 半边数据结构 (Half-Edge Data Structure)	7
3.1 为什么选择 C++	7
3.2 基本类的定义	7
3.3 顶点、边、面的指针关系	8
3.3.1 顶点的半边指针	8
3.3.2 边的半边指针	9
3.3.3 面的半边指针	9
3.4 指针冗余与时空权衡	9
3.5 半边数据结构的图示	9
3.6 网格的容器结构	10
3.7 本章小结	10
4 飞镖数据结构 (Dart-Based Data Structure)	10
4.1 从半边到飞镖：更抽象的视角	10
4.2 相邻关系 (Adjacency)	11
4.3 Alpha 变换	11
4.4 Alpha 变换的几何直观	12
4.5 代数表达与编程实现	12
4.6 本章小结	13
5 群论视角：轨道与生成元	13
5.1 从变换到群作用	13
5.2 生成元与轨道	13
5.3 拓扑元素作为轨道	14
5.4 映射复合的顺序	14
5.5 群作用的几何意义	14
5.6 本章小结	15

6	从数据结构到数值计算	15
6.1	数据结构在工业软件中的位置	15
6.2	稀疏矩阵与刚度矩阵	15
6.3	PDE 求解的线性化	16
6.4	与有限元方法的联系	16
6.5	本章小结	16
7	总结与延伸	17
7.1	讲座核心回顾	17
7.2	核心观点提炼	17
7.3	从理论到代码的价值观	17
7.4	后续课程展望	18
7.5	延伸思考	18
7.6	拓展阅读	18

1 引言：从理论到计算的过渡

1.1 课程背景与目标

本次讲座是“最优传输的理论与计算”系列课程的第九讲，标志着课程从**理论阶段**向**计算阶段**的过渡。主讲人顾险峰教授在第一阶段的八讲中系统介绍了最优传输（Optimal Transportation, OT）的数学理论基础，包括蒙日问题、康托罗维奇松弛、Wasserstein 距离、Brenier 定理等核心内容。

课程两阶段划分

- **第一阶段（第 1–8 讲）**：最优传输的数学理论，侧重概念理解与定理证明。
- **第二阶段（第 9–16 讲）**：最优传输的算法实现，侧重数据结构、编程与数值计算。

顾教授指出，许多具有工科背景的同学反映，对第一阶段的理论内容未能“彻头彻尾”地理解。他强调这属于正常现象——深刻的数学理论需要时间来消化，而**动手实践**是加深理解的最佳途径。因此，第二阶段的课程将着重讲解算法的具体实现，帮助学习者将理论转化为可运行的代码。

1.2 几何编程的核心挑战：数据结构

本次讲座的核心主题是**几何变分算法的数据结构**。顾教授用一个形象的比喻说明其重要性：

“每个行业都有自己的看家本领。对于计算机而言，数据结构是基本功之一。而对于几何编程，数据结构是一个很大很大的难点。”

他以学习拓扑学和几何学为例：学习拓扑的第一个坎是“上同调”（cohomology），学习几何的第一个坎是“黎曼度量”（Riemannian metric），而**几何编程的第一个坎就是数据结构**。

核心观点

几何编程的本质是**用计算机表达曲面**。为此，需要给曲面一个“好用”的数据结构。三角网格（triangular mesh）是表达曲面的标准方式，也称为**单纯复形**（simplicial complex），包含顶点、边、面以及颜色信息。

1.3 工业软件与几何处理

顾教授将本次课程的内容与当前热门的**工业软件**领域联系起来。工业软件的核心技术可归纳为三大块：

1. **网格生成**（Mesh Generation）
2. **数值 PDE 求解**（Numerical PDE）
3. **计算机辅助设计**（CAD/Visualization）

他指出，在计算力学领域，**70% 的时间都花在网格生成上**。网格生成虽然是计算流程中最初始的一步，却是最困难的一步。这凸显了数据结构在几何计算中的基础性地位。

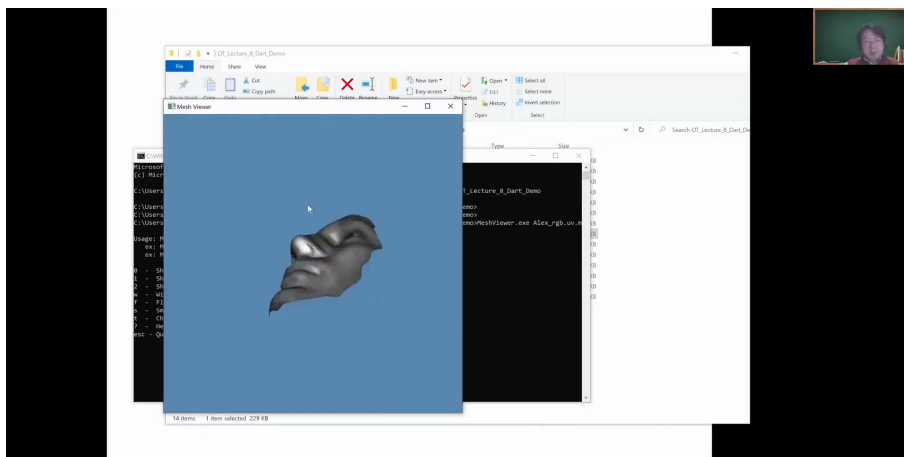


图 1: 三维模型展示：三角网格是计算机表达曲面的标准数据结构。¹

1.4 本章小结

本节介绍了课程的背景与第二阶段的目标：从理论转向计算实现。核心挑战在于**数据结构**的设计—如何用计算机高效、精确地表达和操作几何曲面。三角网格（单纯复形）是标准的数据结构，而工业技术的核心栈（网格生成、数值 PDE、CAD）都建立在这一基础之上。

2 三角网格与最优传输可视化

2.1 三维模型展示

讲座开始时，顾教授通过一系列 demo 展示了三维几何模型的可视化。这些 demo 由陈伟博编写，源代码将开源并分享到课程交流群中。

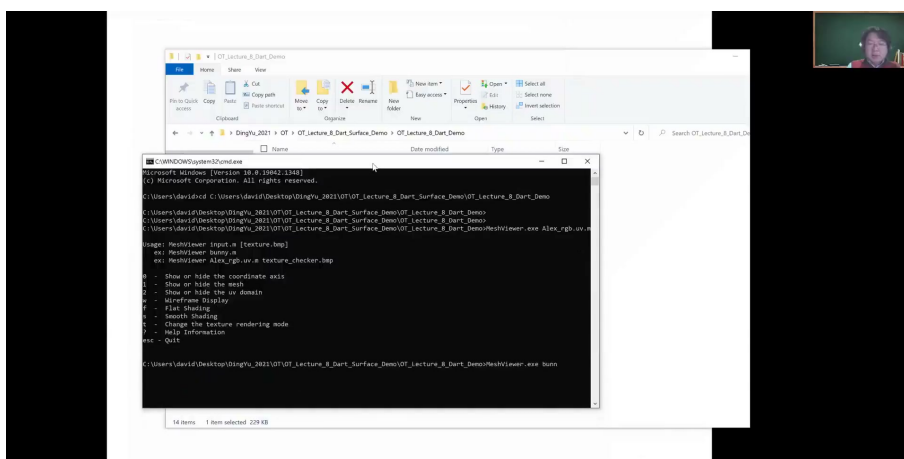


图 2: 斯坦福兔子（Stanford Bunny）：经典的三角网格模型示例。²

展示模型包括：

- **斯坦福兔子（Stanford Bunny）**：计算机图形学中最经典的测试模型之一

¹ 视频画面时间区间：00:03:15–00:03:25。

² 视频画面时间区间：00:04:10–00:04:20。

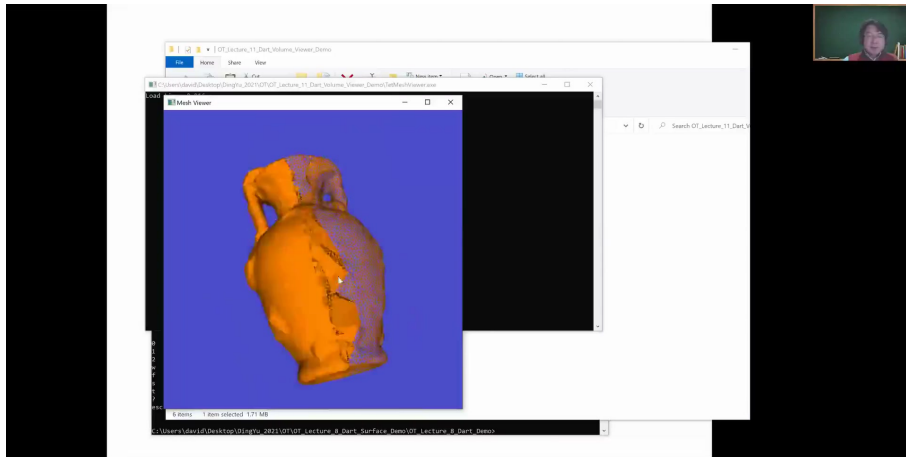


图 4: 切割平面演示：改变切割平面位置观察曲面内部结构。⁴

这个 demo 展示的是一个**模度量模型** (modometric model), 即**四面体网格** (tetrahedral mesh), 不仅仅是表面。当改变切割平面时, 可以观察到曲面内部 (bottom) 的变化。这说明:

曲面与体网格的统一性

曲面和体网格在逻辑上使用的是**同样的数据结构**。这种统一性意味着, 如果掌握了表达曲面的数据结构, 就可以自然地推广到表达三维体 (volume) 的情形。

2.4 最优传输的可视化

讲座中展示了最优传输 (Optimal Transport) 映射的可视化 demo。输入是一张人脸的三角网格, 输出是经过最优传输映射后的结果。

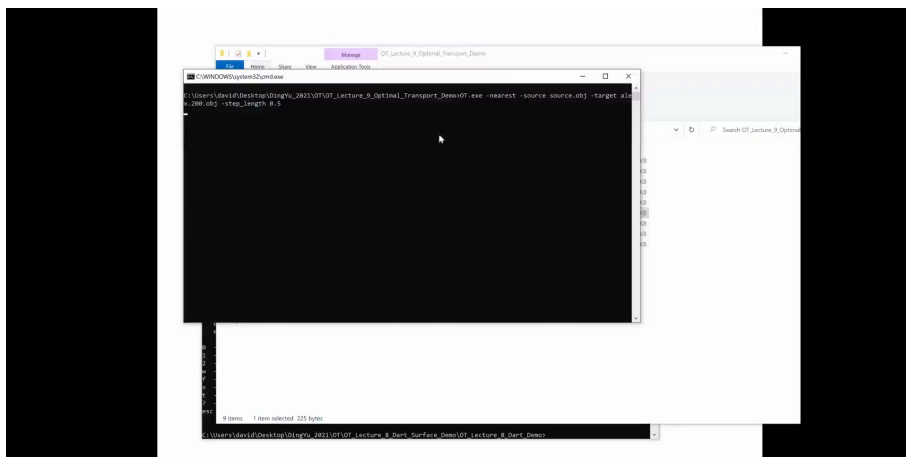


图 5: 最优传输映射可视化：输入人脸网格经过 OT 映射后的结果。⁵

2.5 三角剖分的自适应性

顾教授指出, 三角剖分的质量取决于计算目的:

⁴ 视频画面时间区间: 00:06:30–00:06:45。

⁵ 视频画面时间区间: 00:07:50–00:08:10。

- 如果目的是**有限元计算** (FEM)，三角网格的大小可以**自适应** (adapted) —在曲率大的区域使用更细的网格，在平坦区域使用较粗的网格。
- 如果目的是**可视化**，网格的均匀性可能更重要。

网格生成的自适应策略

在有限元计算中，**自适应网格细化** (adaptive mesh refinement) 是标准技术。其基本思想是：在解的变化剧烈（如高曲率、大梯度）的区域增加网格密度，在变化平缓的区域减少网格密度，以平衡计算精度与效率。

2.6 本章小结

本节通过一系列可视化 demo 展示了三角网格在三维几何处理中的应用。关键要点包括：

1. 三角网格（单纯复形）是表达曲面的标准数据结构
2. 高维网格存在本质困难，需要更抽象的数据结构
3. 曲面与体网格使用统一的数据结构
4. 三角剖分可以根据计算目的进行自适应调整

3 半边数据结构 (Half-Edge Data Structure)

3.1 为什么选择 C++

在正式进入数据结构之前，顾教授解释了为什么选择 C++ 作为几何编程的语言：

“我知道大家都喜欢用 Python，主要是学深度学习出身。但 Python 最终也要被解释成 C，所以还不如直接用 C。因为涉及到几何编程，要大量用指针 (pointer)，而 C++ 是最灵活的。”

C++ 在几何编程中的优势

- **指针操作**：几何编程需要大量指针操作来维护拓扑关系，C++ 的指针机制最为灵活。
- **性能**：所有真正的几何引擎 (geometry engine) 内核都是用 C++ 编写的。
- **动态网格**：在计算过程中三角网格是不断变化的，MATLAB 等语言的矩阵视图难以表达这种动态性。

3.2 基本类的定义

半边数据结构 (Half-Edge Data Structure) 的核心是四个基本类：

1. **顶点类** (Vertex)
2. **半边类** (HalfEdge)
3. **边类** (Edge)

4. 面类 (Face)

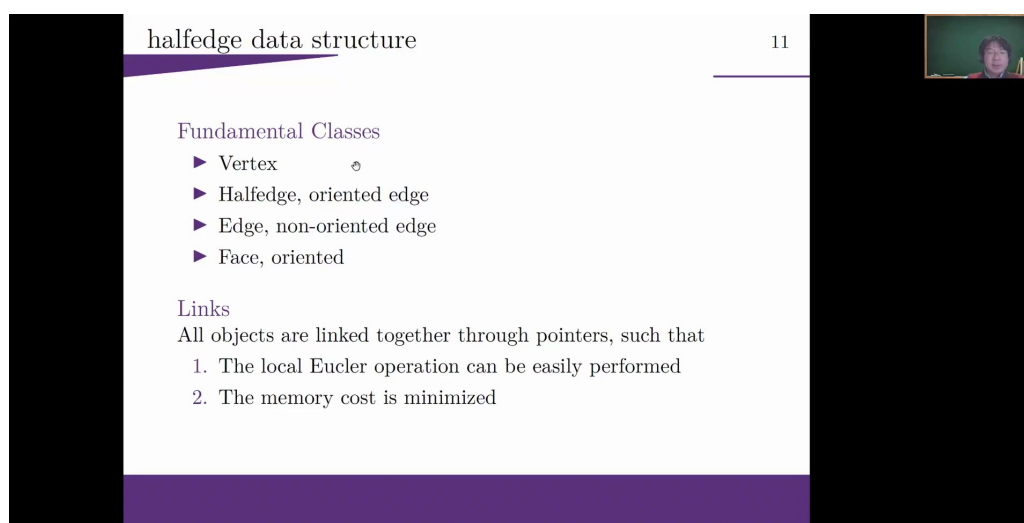


图 6: C++ 基本类定义: Vertex、HalfEdge、Edge、Face 四个核心类。⁶

这些几何单元之间的空间关系用**指针**表示。以半边 (HalfEdge) 为核心对象，每个有向的半边包含以下指针：

表 1: 半边 (HalfEdge) 的指针结构

指针名称	指向对象	说明
target	Vertex	半边的终点顶点
edge	Edge	半边所属的边
face	Face	半边所属的面
next	HalfEdge	同一面中的下一条半边
prev	HalfEdge	同一面中的前一条半边
opposite	HalfEdge	对向的另一半边（同一条边的反向）

3.3 顶点、边、面的指针关系

3.3.1 顶点的半边指针

每个顶点有多个半边指向它，也有多个半边从它出发。在顶点类中，存储一个指针 `halfedge`，指向**以此顶点为终点的一条半边**。

“以一个顶点为终点的半边有很多条，我们选一条，选最逆时针的那一条。”

边界顶点的特殊情况

如果顶点在边界上，选择最逆时针的半边；如果是内部顶点，则随便选择一条即可。边界处理是半边数据结构中最容易出错的地方。

⁶视频画面时间区间：00:20:25–00:20:35。

3.3.2 边的半边指针

每条边如果不是边界边，则有两个半边彼此定向相反，边类中存储指针指向这两个半边。如果是边界边，则只有一个半边。

3.3.3 面的半边指针

每个面（三角形）内部有三条半边，面类中只存储一个指针，指向其中一条半边即可。

3.4 指针冗余与时空权衡

顾教授指出，上述指针结构中包含了**冗余信息**：

“这些指针的信息是有冗余的。理论上你可以去掉一些指针，拓扑信息不会丢失。但那样的话，进行一些局部操作会花时间。”

空间与时间的权衡

半边数据结构中的指针冗余是一种经典的**空间换时间**（space-time tradeoff）策略：

- **保留更多指针**：占用更多内存，但局部操作（如遍历邻域）更快。
- **减少指针**：节省内存，但需要额外计算来推导关系，操作更慢。

实际实现中需要在两者之间找一个**平衡**（balance）。

3.5 半边数据结构的图示

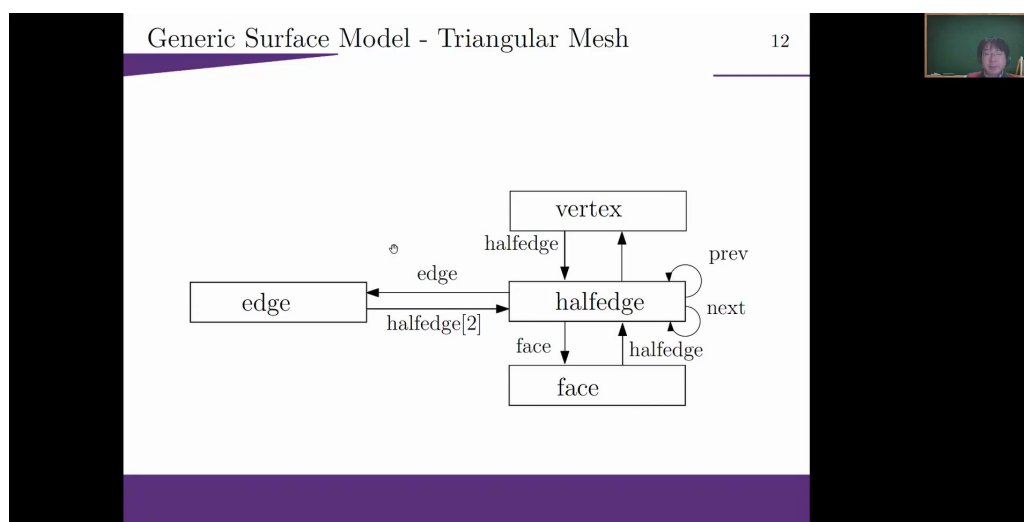


图 7: 半边数据结构示意图：红色半边及其指针关系（next、prev、edge、target、face、opposite）。⁷

图中展示了一个半边（红色）及其周围的指针关系：

- 红色半边有一个指针指向它的 **next** 半边
- 有一个指针指向它的 **prev** 半边

⁷ 视频画面时间区间：00:22:20–00:22:40。

- 有一个指针指向它所依附的 `edge`
- 有一个指针指向它的 `target` 顶点
- 有一个指针指向它所属的 `face`
- 有一个指针指向它的 `opposite` 半边

3.6 网格的容器结构

整个三角网格的数据结构包含以下容器：

```

1 class Mesh {
2     std::vector<Vertex*>    vertices;    // 顶点列表
3     std::vector<HalfEdge*> halfedges;    // 半边列表
4     std::vector<Edge*>     edges;       // 边列表
5     std::vector<Face*>     faces;       // 面列表
6 };

```

Listing 1: 三角网格的容器结构

3.7 本章小结

本节详细介绍了**半边数据结构**—三角网格最常用的数据结构之一。核心要点：

1. 四个基本类：Vertex、HalfEdge、Edge、Face
2. 以半边为核心，通过指针维护拓扑关系
3. 指针结构存在冗余，是空间与时间的权衡
4. 边界处理需要特别注意

4 飞镖数据结构 (Dart-Based Data Structure)

4.1 从半边到飞镖：更抽象的视角

在介绍完半边数据结构后，顾教授引入了一个更抽象、更通用的数据结构—**飞镖** (Dart) 数据结构。这种结构源于**组合地图** (combinatorial map) 理论，能够统一表达不同维度的网格。

飞镖的核心思想

飞镖 (Dart) 是一个元组 (tuple)，包含四个元素：

$$\text{Dart} = (\text{cell}, \text{face}, \text{edge}, \text{vertex})$$

即一个飞镖同时关联了一个**体** (cell)、一个**面** (face)、一条**边** (edge) 和一个**顶点** (vertex)。

4.2 相邻关系 (Adjacency)

两个飞镖之间存在**相邻** (adjacency) 关系, 当且仅当它们的元组中有三个元素相同, 只有一个元素不同。根据哪个元素不同, 定义三种相邻关系:

表 2: 飞镖的相邻关系

名称	相同元素	不同元素	说明
0-相邻	cell, face, edge	vertex	共享同一边和面, 不同顶点
1-相邻	cell, face, vertex	edge	共享同一面和顶点, 不同边
2-相邻	cell, edge, vertex	face	共享同一边和顶点, 不同面
3-相邻	face, edge, vertex	cell	共享同一边、面和顶点, 不同体

“如果两个飞镖只是顶点相同, 面和边都不相同, 这种情况下我们说它们并不相邻。”

4.3 Alpha 变换

在飞镖集合上定义 **Alpha 变换** (alpha operators), 这是飞镖数据结构的核心:

Alpha 变换 (Alpha Operators)

给定一个飞镖 d 和一个维度 $i \in \{0, 1, 2, 3\}$, α_i 作用在 d 上, 将 d 的第 i 维元素替换为与之 i -相邻的飞镖的第 i 维元素。

形式化地说, α_i 是一个映射:

$$\alpha_i : \text{Dart} \rightarrow \text{Dart}$$

Alpha 变换的性质

1. **对合性** (Involution): $\alpha_i^2 = \text{id}$, 即 α_i 的平方等于恒等变换。
2. **唯一性**: 对于封闭流形, 给定一个飞镖, 与之 i -相邻的飞镖存在且唯一。

4.4 Alpha 变换的几何直观

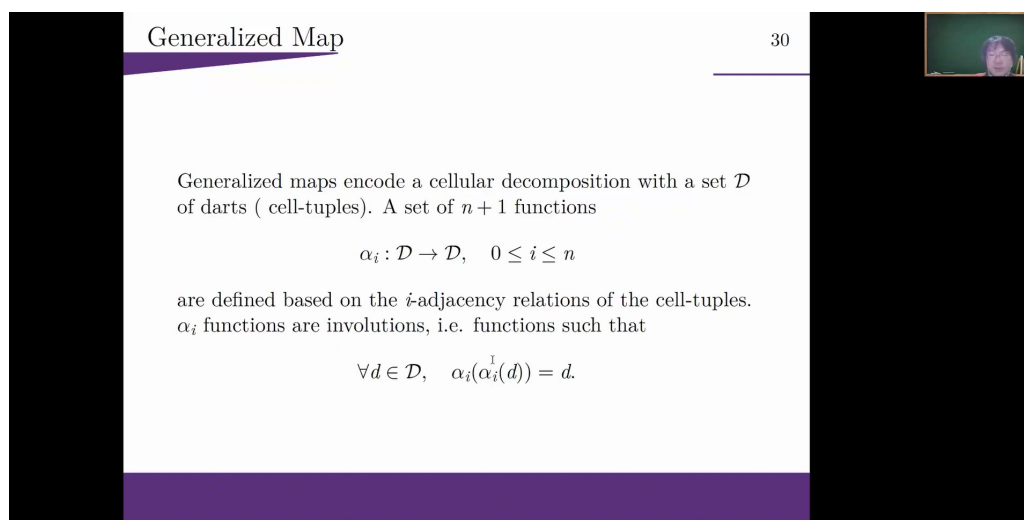


图 8: Alpha 变换的几何直观: α_0 、 α_1 、 α_2 、 α_3 分别对应不同维度的相邻变换。⁸

图中展示了一个四面体 (3-cell) 的剖分, 其中:

- 中间块是一个 **3-cell** (体, 黄色)
- 旁边的面是一个 **2-cell** (面)
- 红色的边是一个 **1-cell** (边)
- 红圈是一个 **0-cell** (顶点)

一个飞镖就是这个块加上这个面、这条边和这个顶点。四个 Alpha 变换的几何意义:

1. α_0 : 保持 cell、face、edge 不变, 将顶点换成相邻的顶点 (沿边翻转)。
2. α_1 : 保持 cell、face、vertex 不变, 将边换成相邻的边 (绕顶点旋转)。
3. α_2 : 保持 cell、edge、vertex 不变, 将面换成相邻的面 (翻面)。
4. α_3 : 保持 face、edge、vertex 不变, 将 cell 换成相邻的 cell (跨体)。

“ α_i 的平方等于自身, 所以它是一个对合 (involution), 是一个自反的变换。”

4.5 代数表达与编程实现

顾教授强调, 飞镖数据结构的优点是它非常**代数化**:

“代数的意思是说容易编程, 计算机容易检验。但我们人类的大脑实际是几何的, 大脑中绝大部分神经元都是用来处理视觉信息, 所以必须要画出来。”

⁸视频画面时间区间: 00:43:50–00:44:15。

代数 vs 几何

- **代数表达**：适合编程实现，计算机可以自动验证正确性。
- **几何直观**：适合人类理解，需要画图来辅助思考。

好的几何数据结构应该同时具备清晰的代数定义和直观的几何解释。

4.6 本章小结

本节介绍了**飞镖数据结构**——一种更抽象、更通用的网格数据结构。核心要点：

1. 飞镖是 (cell, face, edge, vertex) 四元组
2. 相邻关系根据元组中哪个元素不同来分类
3. Alpha 变换 α_i 是核心操作，满足 $\alpha_i^2 = \text{id}$
4. 代数化定义便于编程，但需要几何直观来辅助理解

5 群论视角：轨道与生成元

5.1 从变换到群作用

在飞镖数据结构中，Alpha 变换可以进一步抽象为**群作用** (group action)。这是将代数拓扑与数据结构深度结合的关键一步。

群论视角的核心思想

将飞镖集合上的变换视为**群作用** (group action)：

- 每个 α_i ($i \geq 1$) 是群的一个**生成元** (generator)。
- 生成元生成的群在飞镖集合上的**轨道** (orbit) 定义了网格的拓扑元素。

5.2 生成元与轨道

定义生成元：

- $f_1 = \alpha_1$ ：沿边翻转
- $f_2 = \alpha_2$ ：翻面
- $f_3 = \alpha_3$ ：跨体

注意 α_0 **不包含**在生成元中，因为顶点是由其他生成元的复合作用来定义的。

轨道 (Orbit)

给定一个群 G 作用在集合 X 上，元素 $x \in X$ 的**轨道** (orbit) 定义为：

$$\text{Orb}_G(x) = \{g \cdot x \mid g \in G\}$$

即 x 在群作用下所有可能到达的元素集合。

5.3 拓扑元素作为轨道

在飞镖数据结构中，各种拓扑元素被定义为轨道：

表 3: 拓扑元素的轨道定义

拓扑元素	生成子群	不变量
顶点 (Vertex)	$\langle f_1, f_2 \rangle$	保持顶点不变
边 (Edge)	$\langle f_2, f_3 \rangle$	保持边不变
面 (Face)	$\langle f_1, f_3 \rangle$	保持面不变
体 (Cell)	$\langle f_1, f_2 \rangle$	保持体不变

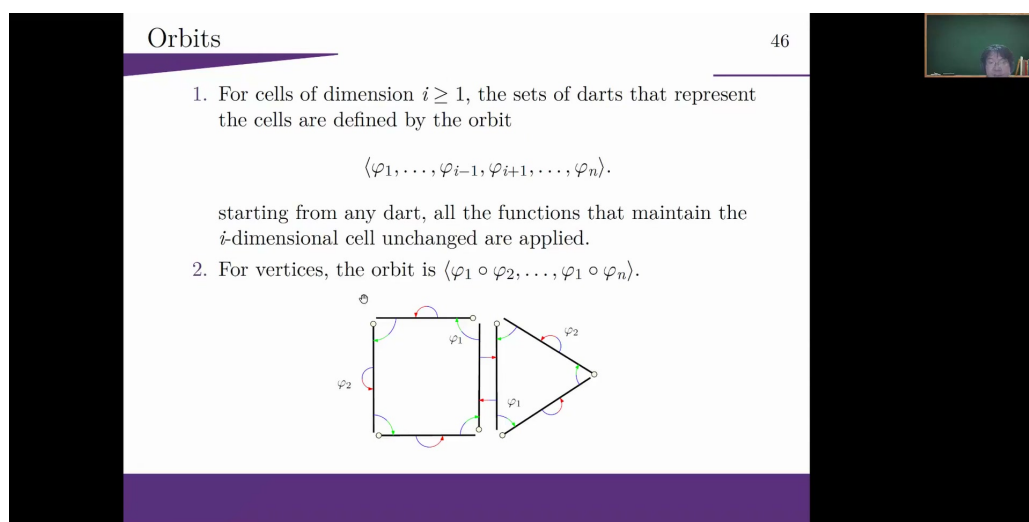


图 9: 群作用与轨道: f_1 和 f_2 作用在飞镖上的轨道定义了一个顶点。⁹

5.4 映射复合的顺序

在群论中，映射复合的顺序需要特别注意：

“ $f_1 \circ f_2$ 代表我们先作用 f_2 ，再作用 f_1 。这个顺序不要搞颠倒。”

映射复合的顺序

在群论中，群作用的写法与映射复合的写法一致：

$$f_1 \circ f_2(x) = f_1(f_2(x))$$

即从右向左作用：先作用 f_2 ，再作用 f_1 。虽然每个变换都是对合 ($f_i^2 = \text{id}$)，在某些情况下颠倒顺序相当于求逆，影响不大，但严格来说应遵守规定的顺序。

5.5 群作用的几何意义

f_1 和 f_2 生成的子群作用在飞镖集合上，保持每个飞镖对应的顶点不变：

⁹视频画面时间区间：01:05:15–01:05:35。

1. 作用 f_2 : 将半边沿其顶点“旋转”到相邻位置。
2. 再作用 f_1 : 将边“旋转”到相邻位置。
3. 因此 $f_1 \circ f_2$ 的作用保持顶点不变, 定义了一个顶点。

5.6 本章小结

本节从群论视角重新诠释了飞镖数据结构。核心要点:

1. Alpha 变换 α_i ($i \geq 1$) 是群的生成元
2. 拓扑元素(顶点、边、面、体)被定义为群作用的轨道
3. 映射复合的顺序是从右向左
4. 群论视角提供了统一的代数框架, 适用于任意维度的网格

6 从数据结构到数值计算

6.1 数据结构在工业软件中的位置

顾教授在讲座末尾总结了数据结构在整个计算流程中的位置:

“用大的数据结构, 把大的数据结构变成矩阵之后进行优化计算。用大的数据结构把它变成稀疏矩阵 (sparse matrix), 这时我们要在流形上解 PDE 方程, 就跟经典的有限元计算差不多。”

从几何到代数的完整流程

1. **几何模型**: 三维扫描或 CAD 建模得到曲面
2. **网格生成**: 将曲面三角剖分, 得到三角网格
3. **数据结构**: 用半边或飞镖数据结构表达网格拓扑
4. **稀疏矩阵**: 从数据结构导出稀疏刚度矩阵 (stiffness matrix)
5. **PDE 求解**: 将泊松方程、拉普拉斯方程等化为线性系统求解

6.2 稀疏矩阵与刚度矩阵

通过三角剖分, 可以得到**刚度矩阵** (stiffness matrix)。这个矩阵具有以下特点:

- **稀疏性** (Sparsity): 每个顶点通常只有 5-6 个邻接顶点, 因此矩阵每行只有 5-6 个非零元素。
- **对称性**: 对于自伴算子 (如拉普拉斯算子), 刚度矩阵是对称的。
- **正定性**: 对于椭圆型 PDE, 刚度矩阵是正定的。

“每一行只有五六个非零元素, 其他都是零。这和用连接矩阵 (adjacency matrix) 表达的矩阵概念是完全不一样的。”

6.3 PDE 求解的线性化

最终，任何几何量或物理量的计算都归结为求解偏微分方程（PDE）：

PDE 求解的归约

1. 求任何一个物理量或几何量
2. 最终归结为求解偏微分方程
3. PDE 通过线性化归结为求解线性矩阵

这是整个工业界计算的基础框架。

6.4 与有限元方法的联系

顾教授指出，上述流程与经典的有限元方法（Finite Element Method, FEM）非常相似：

- 三角剖分 → 有限元网格
- 刚度矩阵 → 有限元刚度矩阵
- 泊松方程/拉普拉斯方程 → 标准的椭圆型 PDE

有限元方法回顾

有限元方法（FEM）是求解偏微分方程的数值方法，基本步骤为：

1. 将求解域剖分为有限个单元（如三角形、四面体）
2. 在每个单元上选择适当的基函数（通常为多项式）
3. 将 PDE 的弱形式离散化为代数方程组
4. 组装全局刚度矩阵和载荷向量
5. 求解线性系统

6.5 本章小结

本节总结了数据结构在数值计算流程中的位置：

1. 数据结构是连接几何模型与数值计算的桥梁
2. 从数据结构可以导出稀疏刚度矩阵
3. PDE 求解最终归结为线性系统的求解
4. 整个流程与经典有限元方法高度一致

7 总结与延伸

7.1 讲座核心回顾

本次讲座（最优传输系列第九讲）的核心内容是**几何变分算法的数据结构**。顾险峰教授从三个层次递进地介绍了这一主题：

1. **直观层面**：通过大量可视化 demo 展示了三角网格在三维几何处理中的应用，包括斯坦福兔子、纹理映射、最优传输映射等。
2. **实现层面**：详细介绍了**半边数据结构** (Half-Edge Data Structure) 的 C++ 实现，包括 Vertex、HalfEdge、Edge、Face 四个基本类及其指针关系。
3. **抽象层面**：引入了**飞镖数据结构** (Dart-Based Data Structure) 和 **Alpha 变换**，从群论视角将拓扑元素定义为群作用的轨道，提供了统一表达任意维度网格的代数框架。

7.2 核心观点提炼

本次讲座的核心观点

1. **数据结构是几何编程的第一坎**：比上同调、黎曼度量更基础，是进入几何计算的必经之路。
2. **半边结构是工程标准**：Vertex-HalfEdge-Edge-Face 四类加指针，是表达三角网格最实用的数据结构。
3. **飞镖结构是理论统一**： $\text{Dart} = (\text{cell}, \text{face}, \text{edge}, \text{vertex}) + \text{Alpha 变换}$ ，可统一表达任意维度单纯复形。
4. **群论视角提供高维推广**：将拓扑元素定义为群作用的轨道，是高维网格数据结构的关键抽象。

7.3 从理论到代码的价值观

顾教授在讲座结尾分享了一个重要的价值观：

“在座有很多同学是学计算几何的，所以大家肯定清楚，在计算几何领域中，绝大多数的算法论文发表之后，实际是根本没办法实现，只是纸上谈兵。但如果在工业界，真的做元宇宙、做自动驾驶，需要实打实地把代码写出来。”

“The idea is cheap, show me the code. 真的要把代码写出来。”

数学家与计算机科学家的价值观差异

- **数学家**：关注定理的存在性、唯一性、收敛性证明，代码实现是次要问题。
- **计算机科学家/工程师**：关注算法的可实现性、效率、鲁棒性，“show me the code”是最终标准。

顾教授强调，本系列课程的目标是后者——不仅停留在理论层面的计算，而是要实打实地把代码写出来。

7.4 后续课程展望

顾教授预告了下一次讲座（周四）的内容：

- **下一次讲座**：将讲解具体的算法，难度会比本次讲座低很多。
- **后续主题**：将涉及 Ricci 流（Ricci flow）的可视化，同样使用飞镖数据结构。
- **源代码**：所有 demo 的源代码将开源分享到课程群中。

学习建议

顾教授特别强调：“今天讲的比较难以理解，是大家要仔细看。视频还有 demo 开源，开源代码会发到群里面。大家读了代码之后，会真正理解。如果不读代码的话，是很难理解的。”

7.5 延伸思考

1. **高维最优传输**：本次讲座介绍的数据结构如何推广到高维情形？飞镖数据结构的群论视角为此提供了理论基础。
2. **稀疏矩阵求解器**：从数据结构导出的稀疏刚度矩阵需要专门的稀疏线性求解器（如共轭梯度法、多重网格法），这是数值计算的核心。
3. **3D 打印与自由曲面透镜**：讲座中提到的“任意形状透镜”是 3D 打印领域的热点，最优传输映射在光学设计中具有重要应用。
4. **深度学习与几何**：最优传输理论为深度学习中的概率分布学习、生成模型（如 GAN、扩散模型）提供了坚实的数学基础。

7.6 拓展阅读

- Gu, X., Yau, S.-T. *Computational Conformal Geometry*. International Press, 2008.
- Lévy, B., Zhang, H. *Spectral Geometry Processing*. SIGGRAPH Course Notes.